

2.2 User Interfaces

Navigation

Objectives

Understand navigation principles

Build a dynamic and multi-pane UI using Fragment

Create Navigation Component

Create Dialog

Create Navigation Drawer

Create Tabs

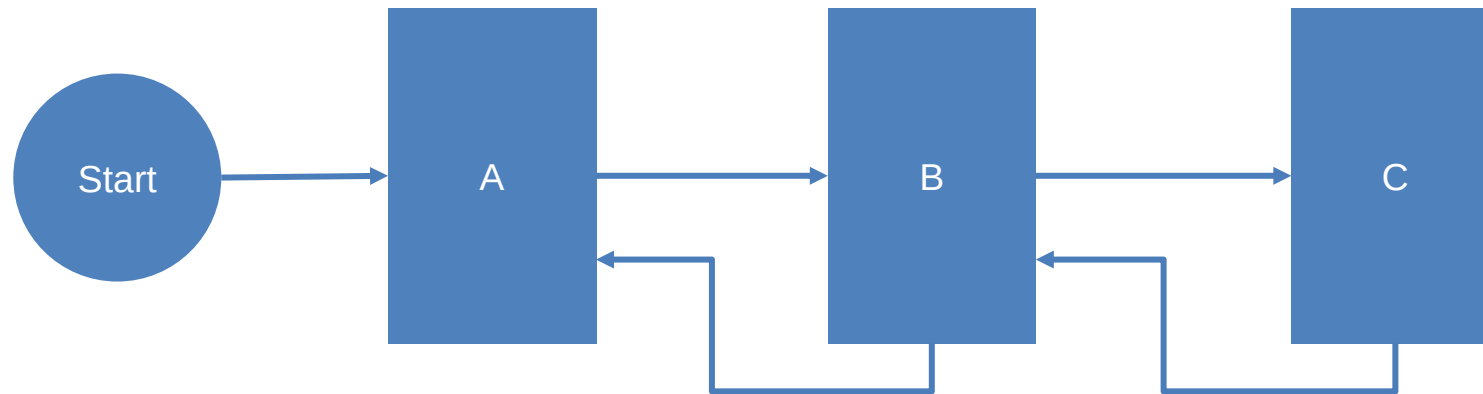
Navigation principles

Navigation Principles

1. Fixed start destination
2. Navigation state is represented as a stack of destinations
3. Up and Back are identical within your app's task
4. The Up button never exits your app
5. Deep linking simulates manual navigation

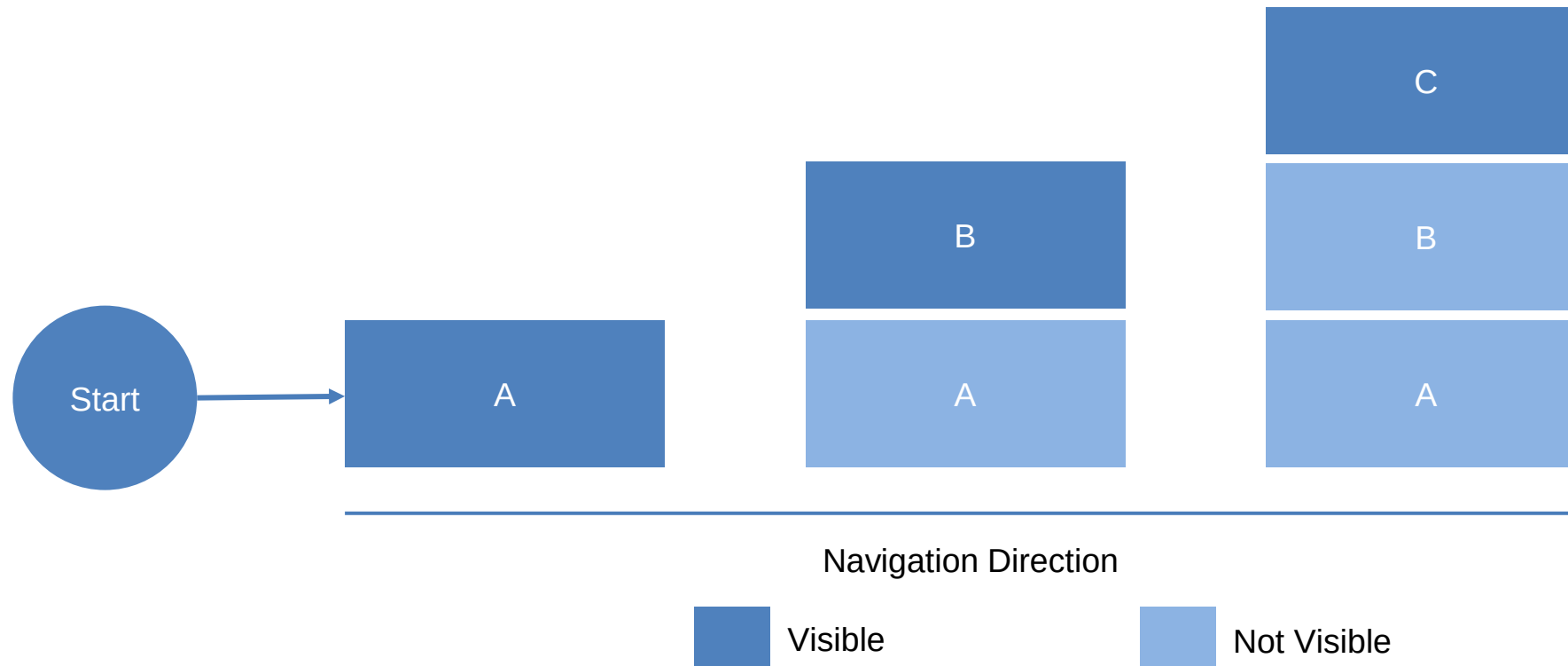
Navigation Principles

1. Fixed start destination



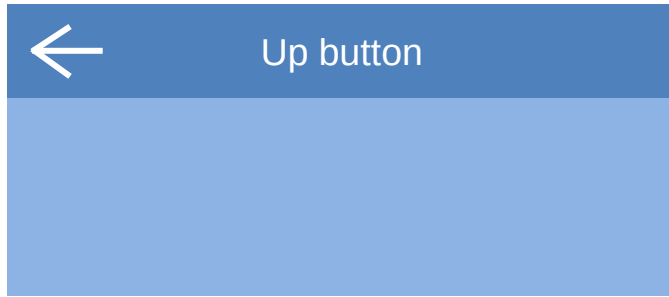
Navigation Principles

2. Navigation state is represented as a stack of destinations



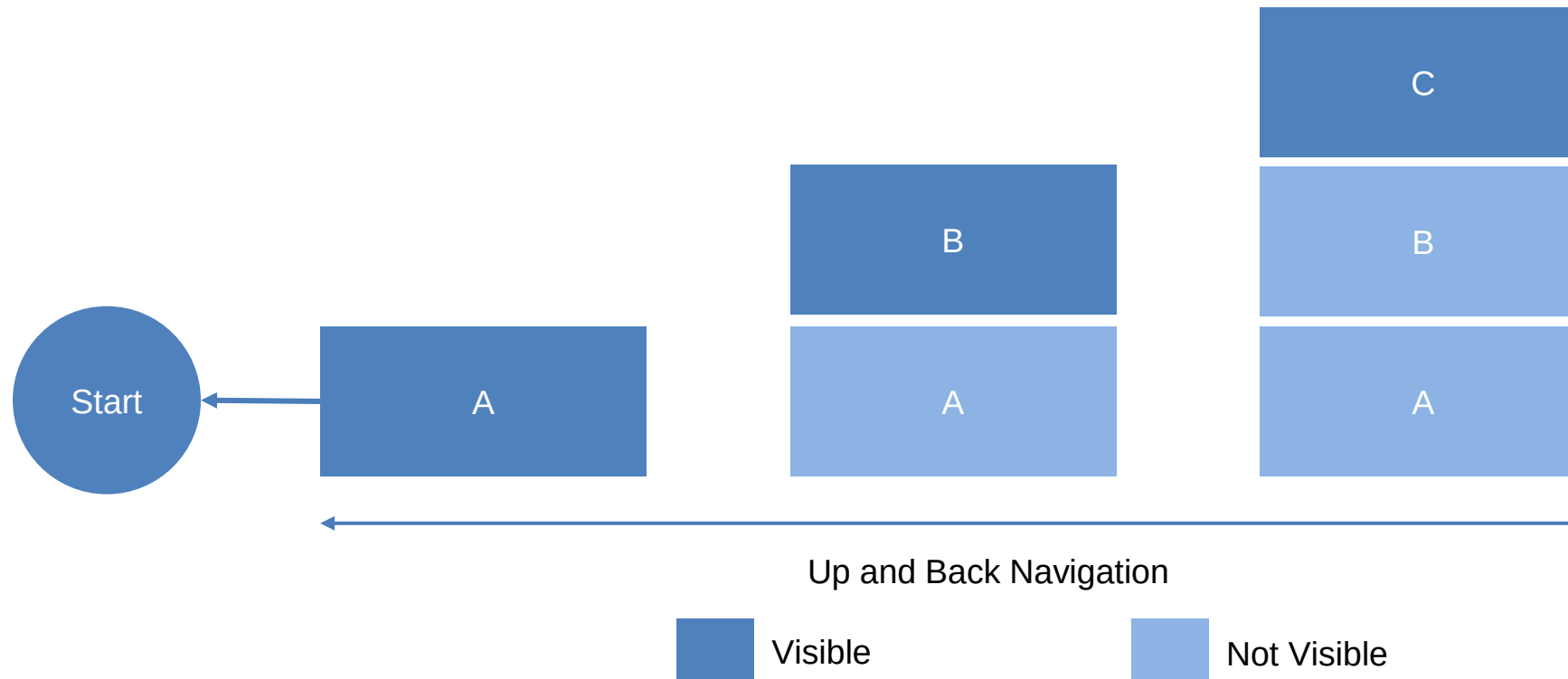
Navigation Principles

3. Up and Back are identical within your app's task



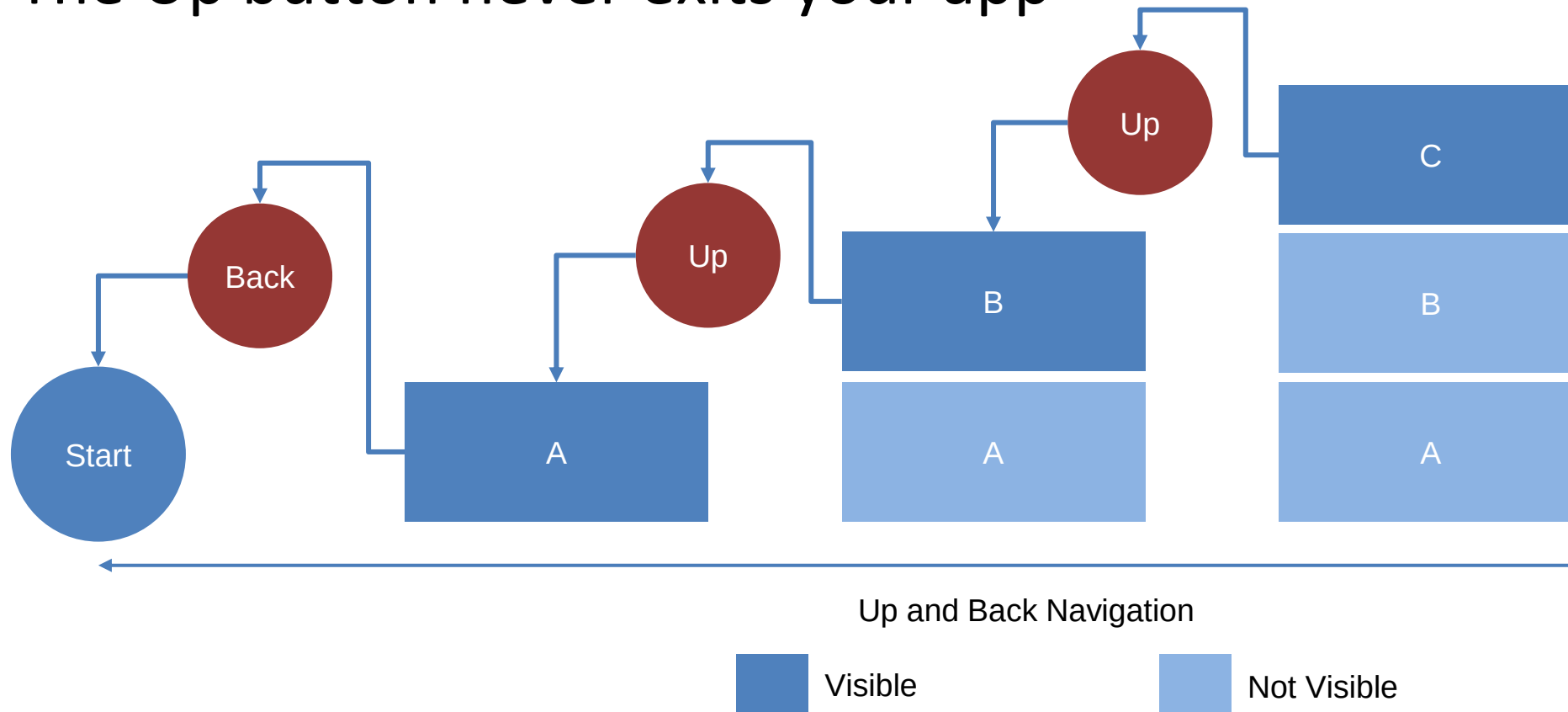
Navigation Principles

3. Up and Back are identical within your app's task



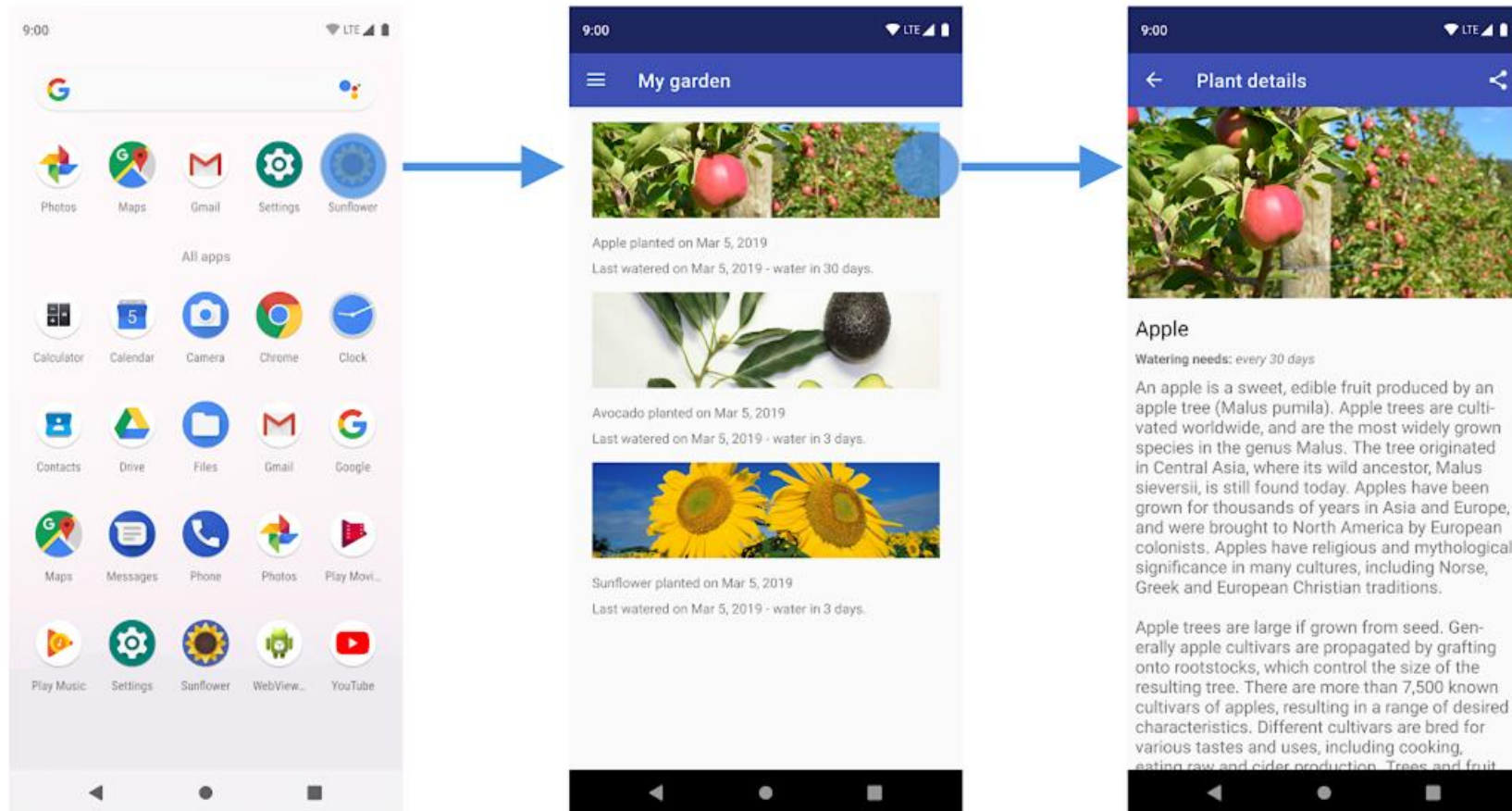
Navigation Principles

4. The Up button never exits your app



Navigation Principles

5. Deep linking simulates manual navigation

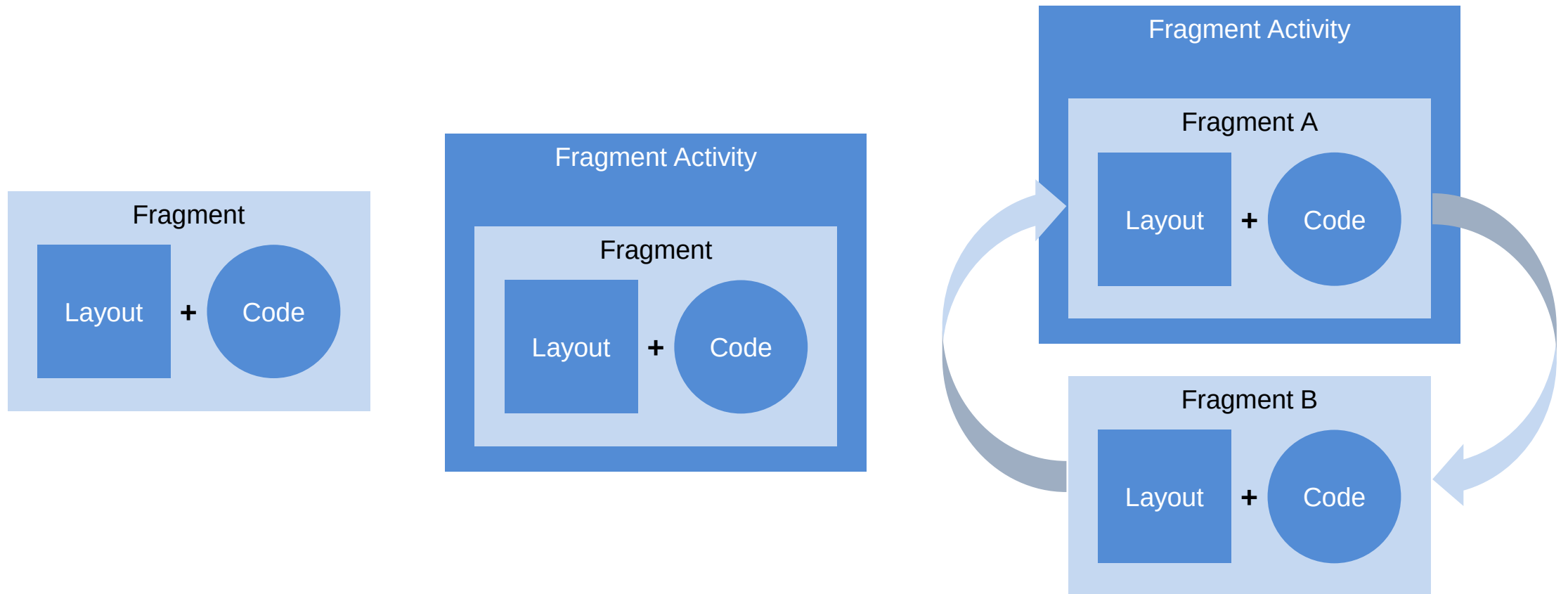


Question?

1. Why the behaviour of the Back and Up buttons must be different?

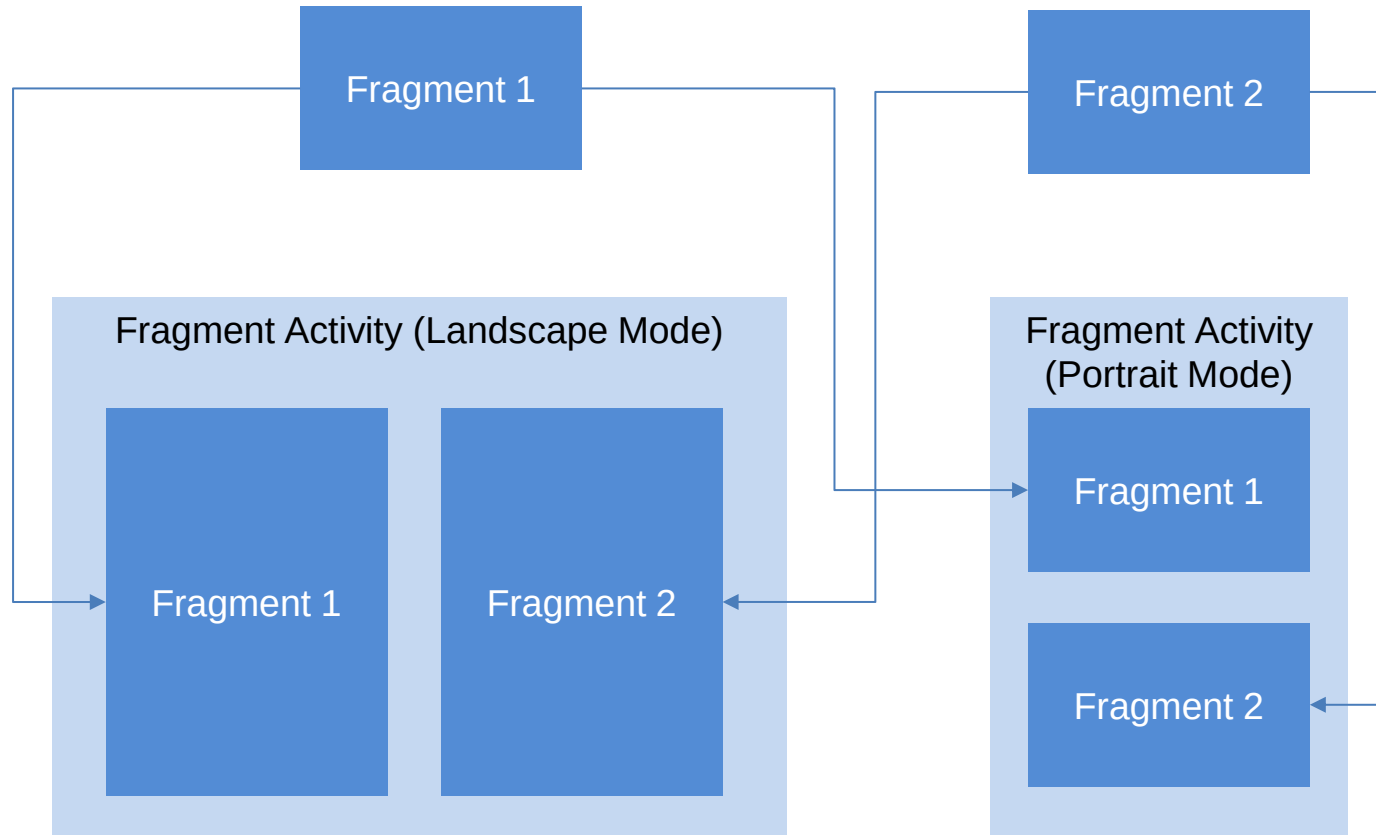
Build a dynamic and multi-pane UI using Fragment

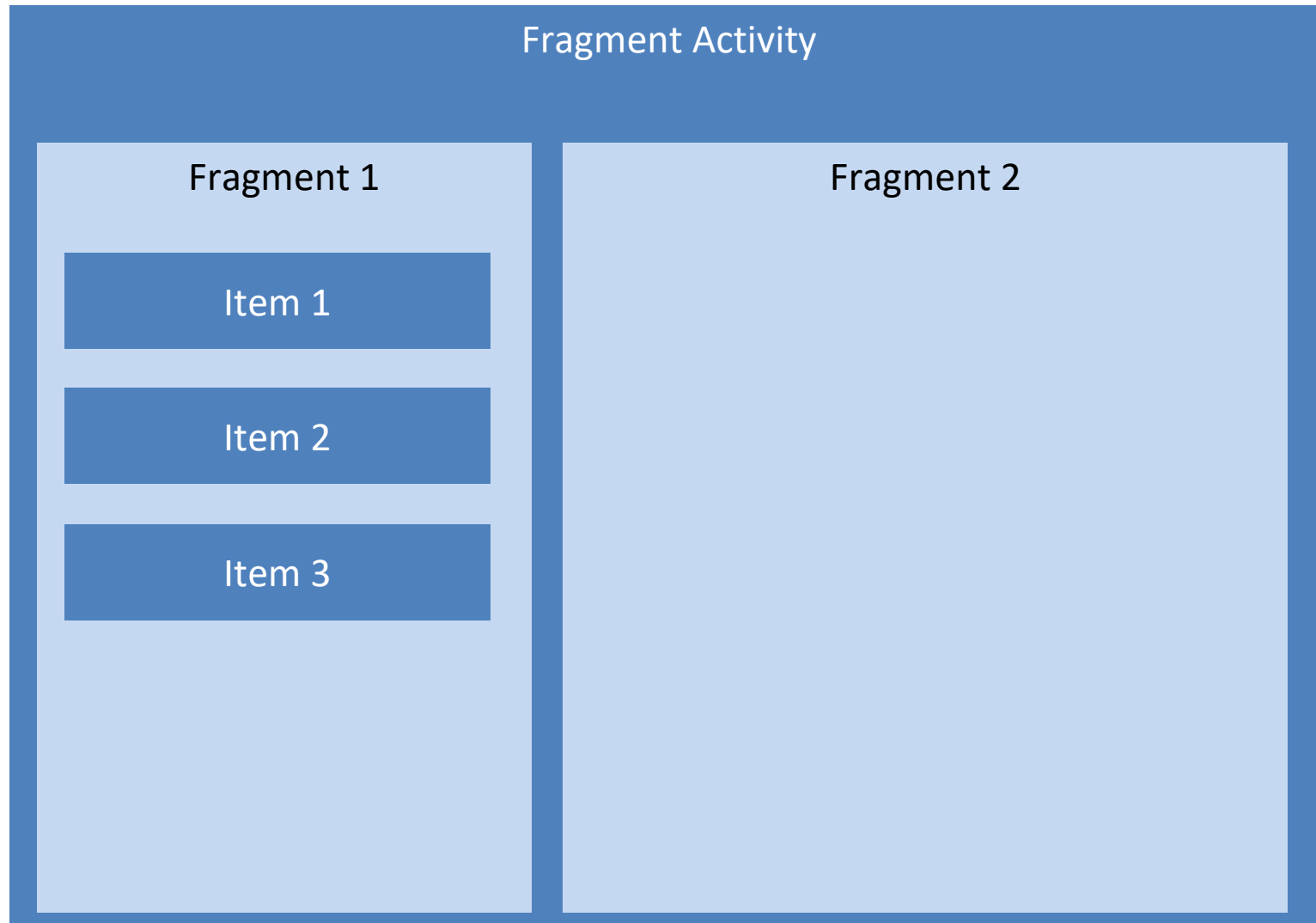
Fragment



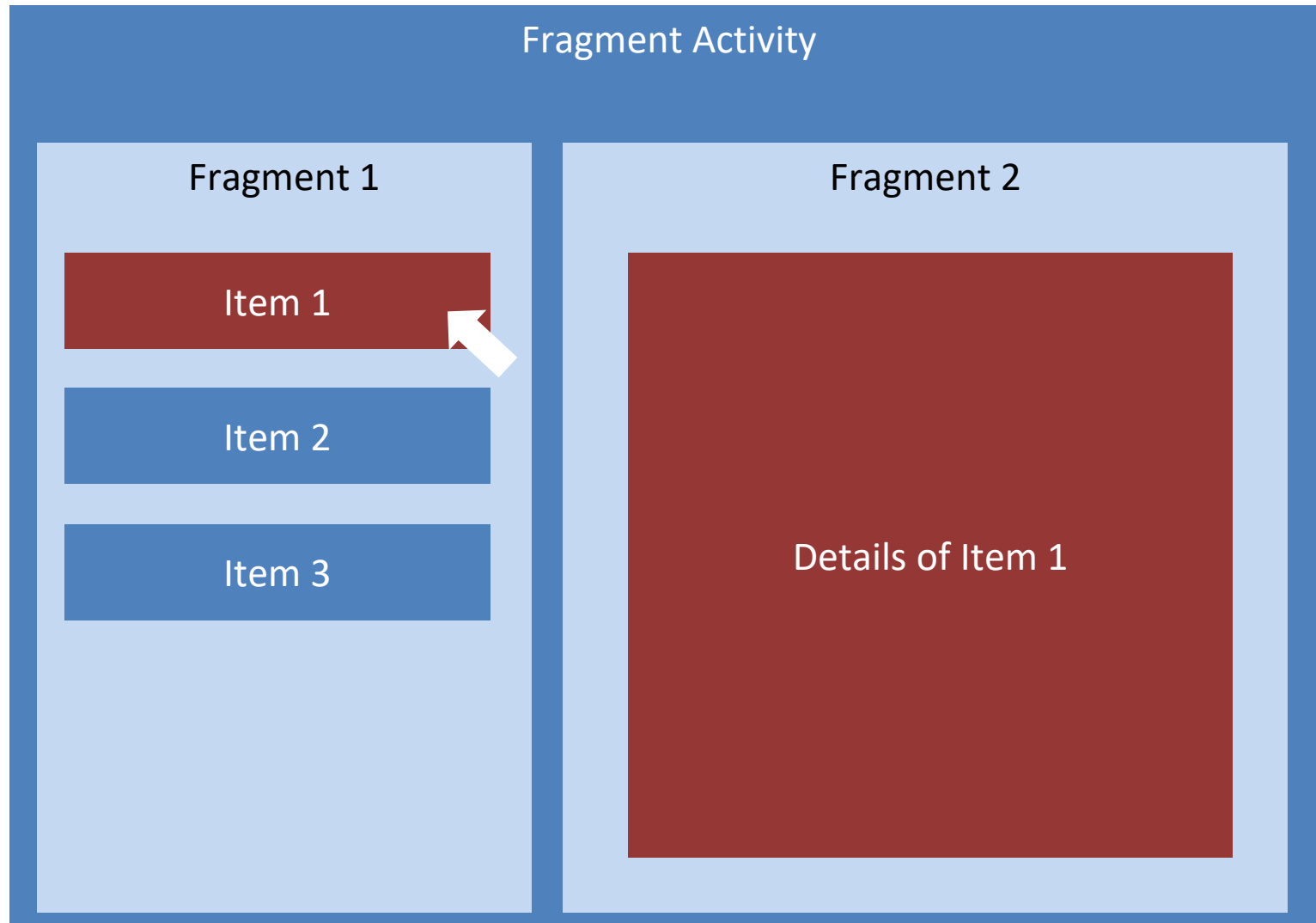
Fragment

An example of how two UI modules defined by fragments

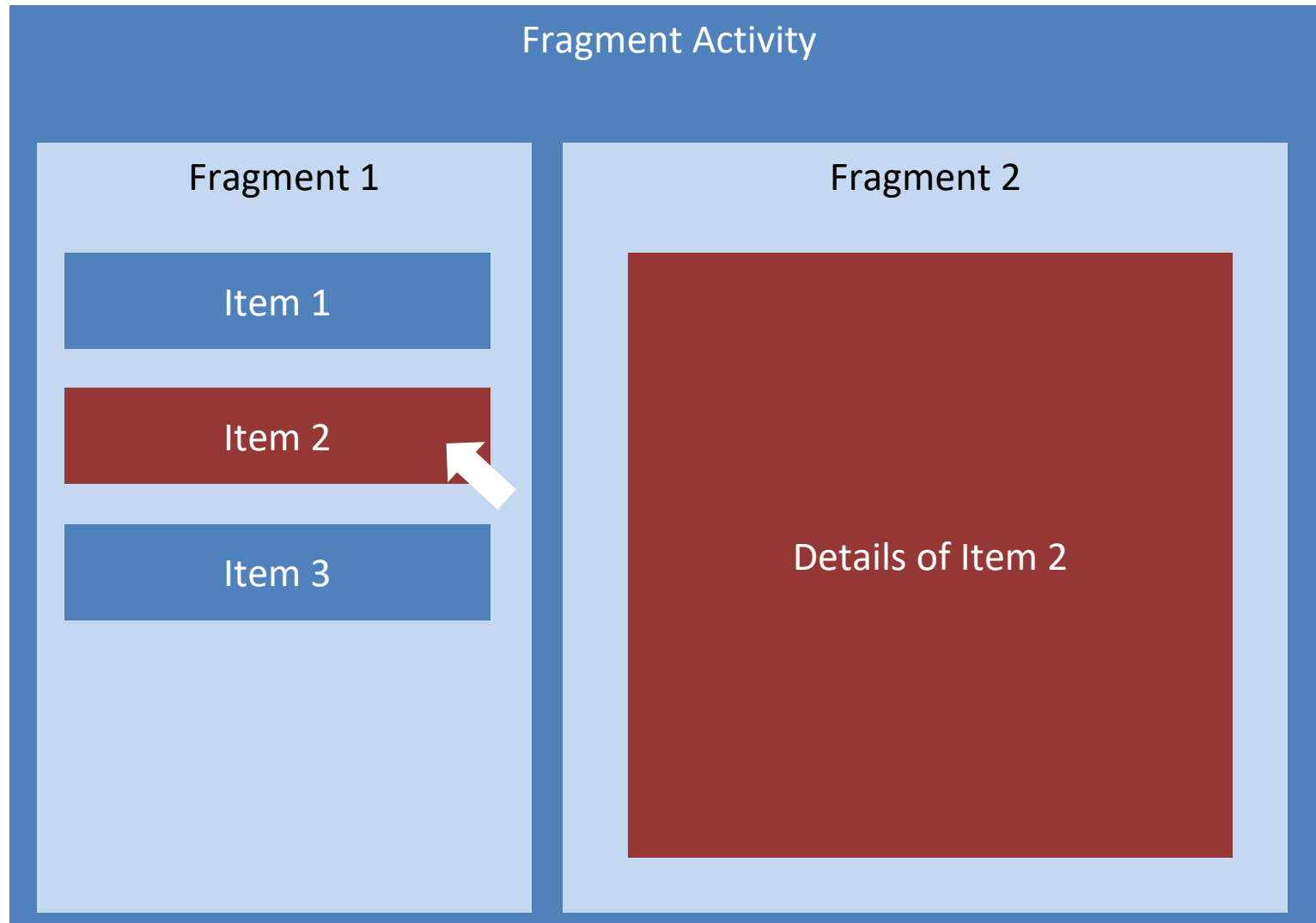




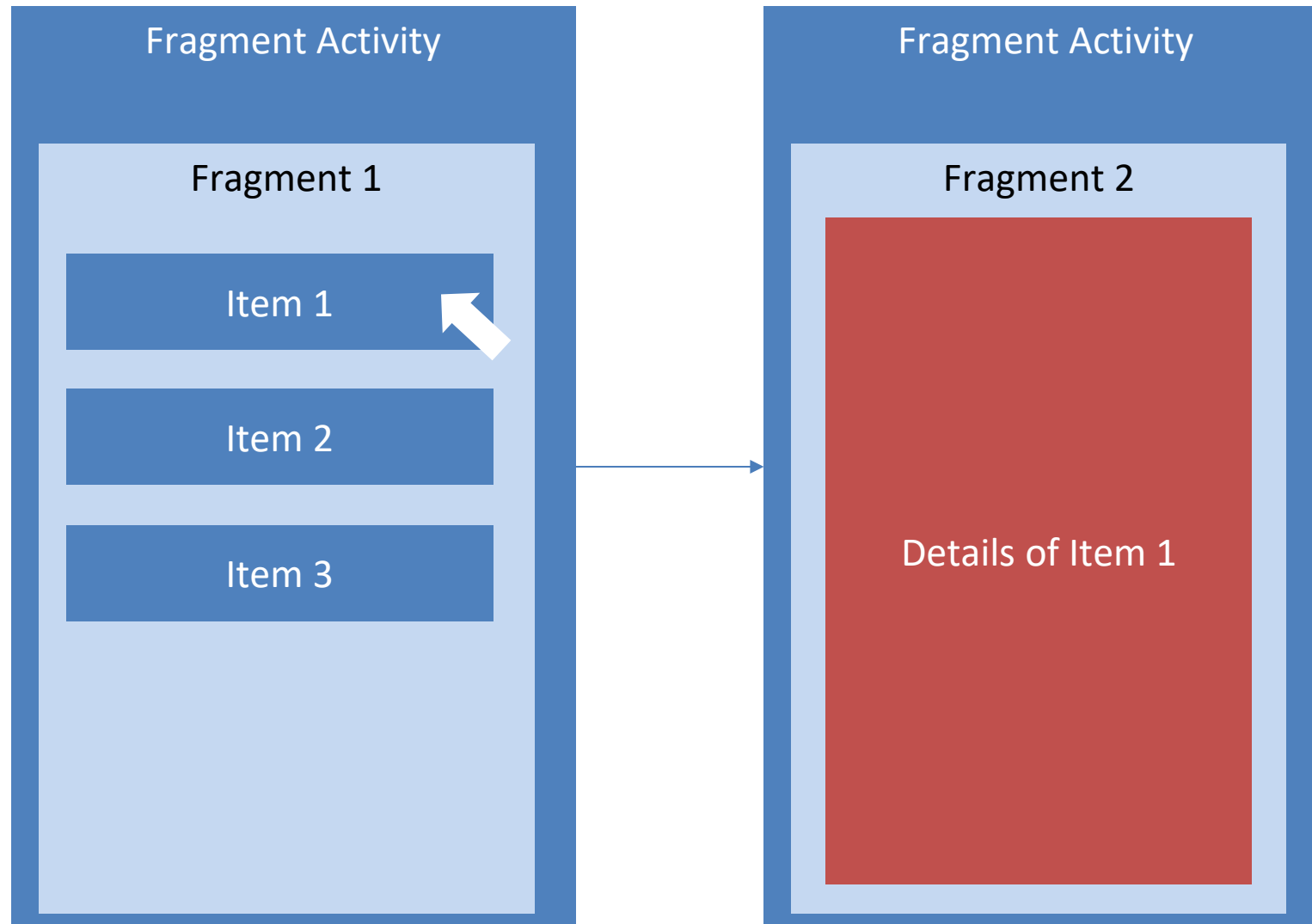
Landscape Mode



Landscape Mode

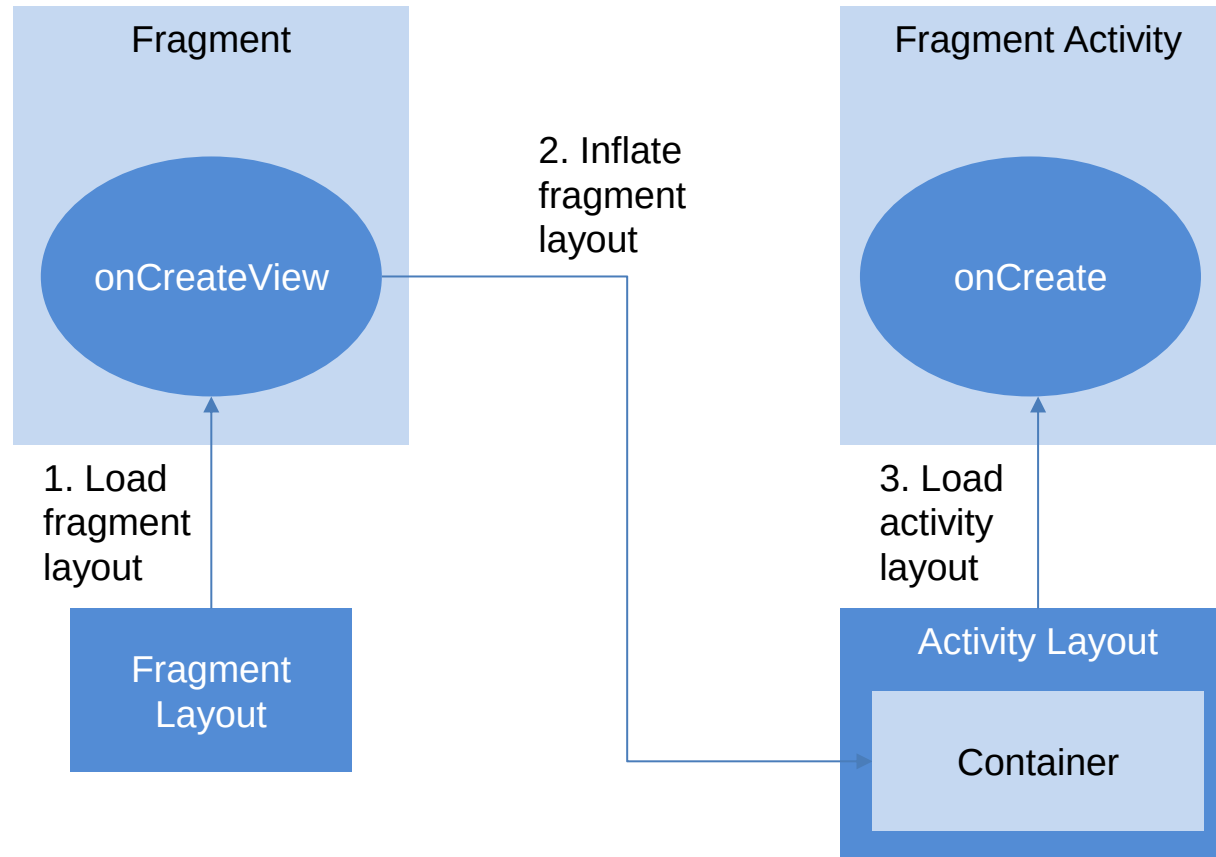


Landscape Mode



Portrait Mode

Add layout (UI)

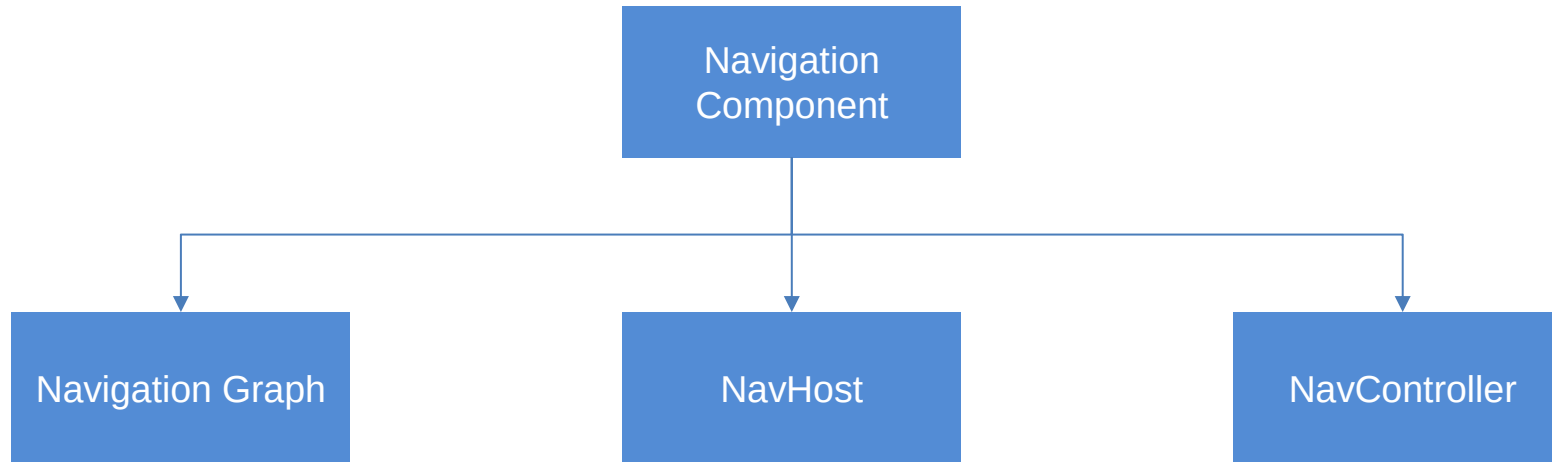


Jetpack Compose vs Fragments

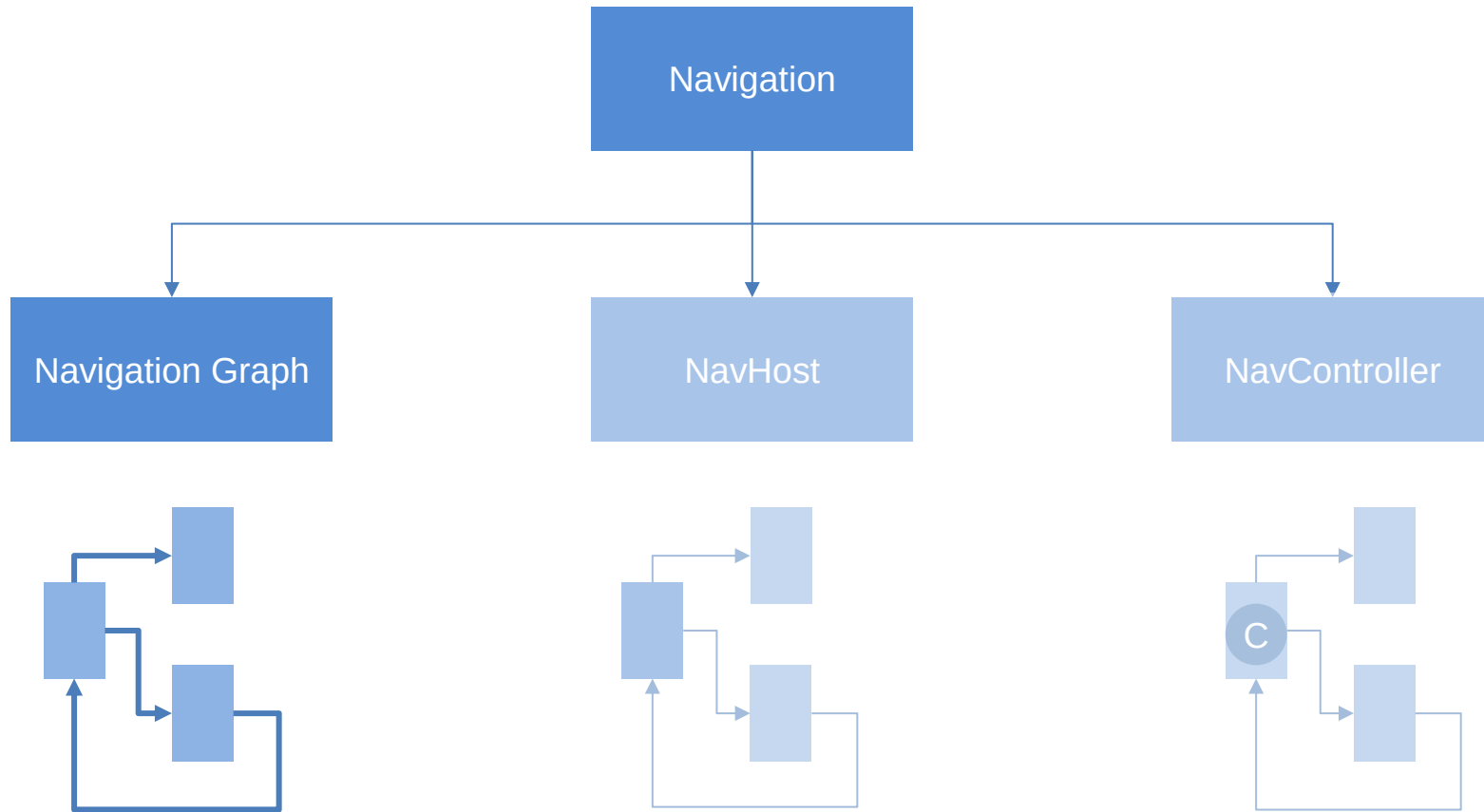
Feature	Fragments (Old)	Jetpack Compose (New)
UI Definition	XML Layouts + Kotlin	Pure Kotlin Composables
Navigation	FragmentManager + NavController	NavHost + NavController
Performance	Slower (View Inflation)	Faster (Composables are lightweight)
State Handling	ViewModel + SavedStateHandle	remember + State
Lifecycle Complexity	Fragment Lifecycle (onCreate, onStart...)	Simpler Lifecycle with Composables

Create Navigation Component

Navigation Component (Old Approach)



Navigation



XML resource that contains all navigation-related information

<https://developer.android.com/guide/navigation>

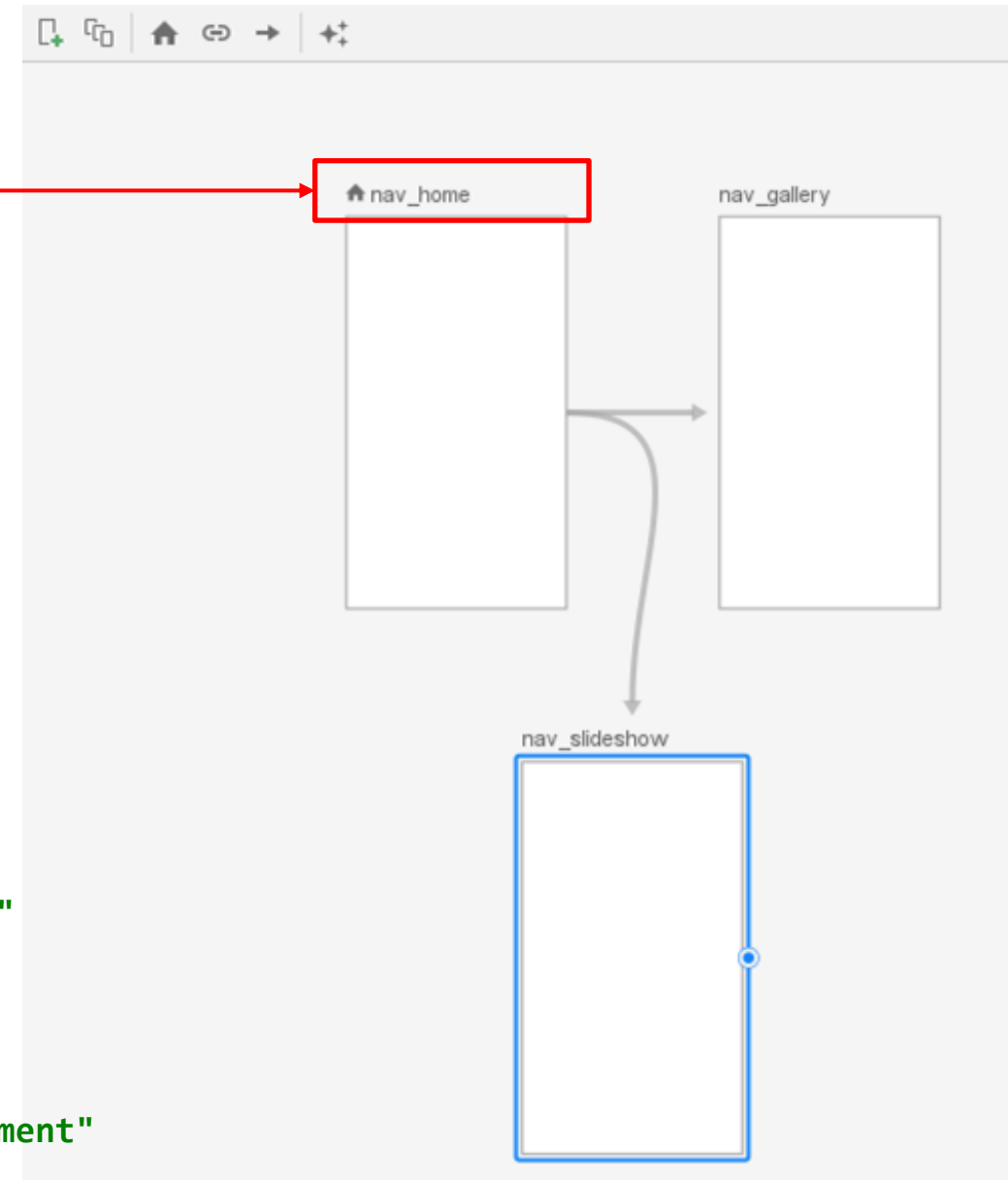
```

<?xml version="1.0" encoding="utf-8"?>
<navigation
    xmlns:android="http://schemas.android.com/apk/res/android"
    . . .
    app:startDestination="@+id/nav_home">
    <fragment
        android:id="@+id/nav_home"
        android:name="my.tarc.myapplication.ui.home.HomeFragment"
        . . .>
        <action
            android:id="@+id/homeToGallery"
            app:destination="@id/nav_gallery" />
        <action
            android:id="@+id/homeToSlideShow"
            app:destination="@id/nav_slideshow" />
        </fragment>

    <fragment
        android:id="@+id/nav_gallery"
        android:name="my.tarc.myapplication.ui.gallery.GalleryFragment"
        . . ./>

    <fragment
        android:id="@+id/nav_slideshow"
        android:name="my.tarc.myapplication.ui.slideshow.SlideshowFragment"
        . . ./>
    </navigation>

```



```

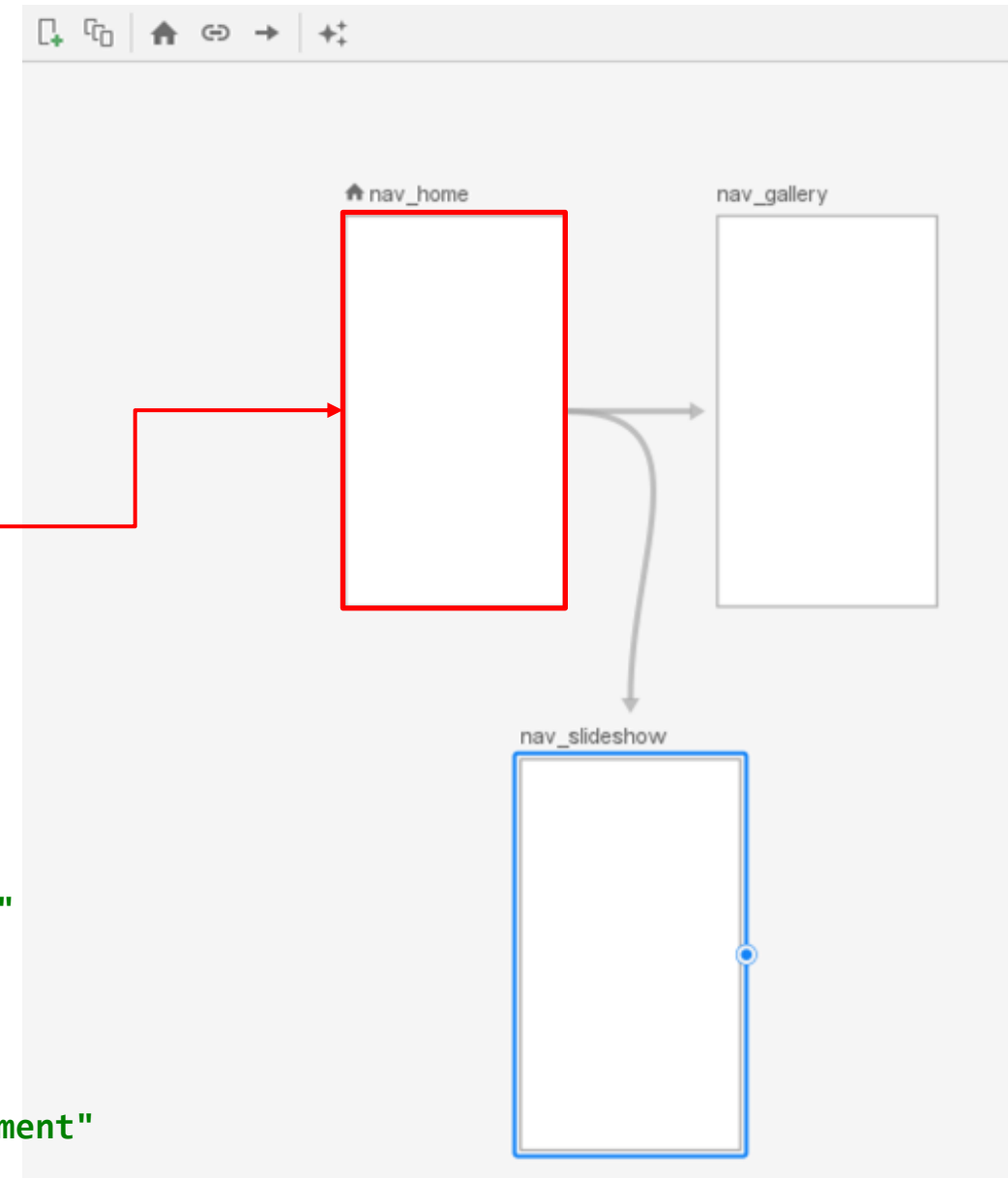
<?xml version="1.0" encoding="utf-8"?>
<navigation
    xmlns:android="http://schemas.android.com/apk/res/android"
    . . .
    app:startDestination="@+id/nav_home">

    <fragment
        android:id="@+id/nav_home"
        android:name="my.tarc.myapplication.ui.home.HomeFragment"
        . . .>
        <action
            android:id="@+id/homeToGallery"
            app:destination="@id/nav_gallery" />
        <action
            android:id="@+id/homeToSlideShow"
            app:destination="@id/nav_slideshow" />
        </fragment>

    <fragment
        android:id="@+id/nav_gallery"
        android:name="my.tarc.myapplication.ui.gallery.GalleryFragment"
        . . ./>

    <fragment
        android:id="@+id/nav_slideshow"
        android:name="my.tarc.myapplication.ui.slideshow.SlideshowFragment"
        . . ./>
</navigation>

```



```

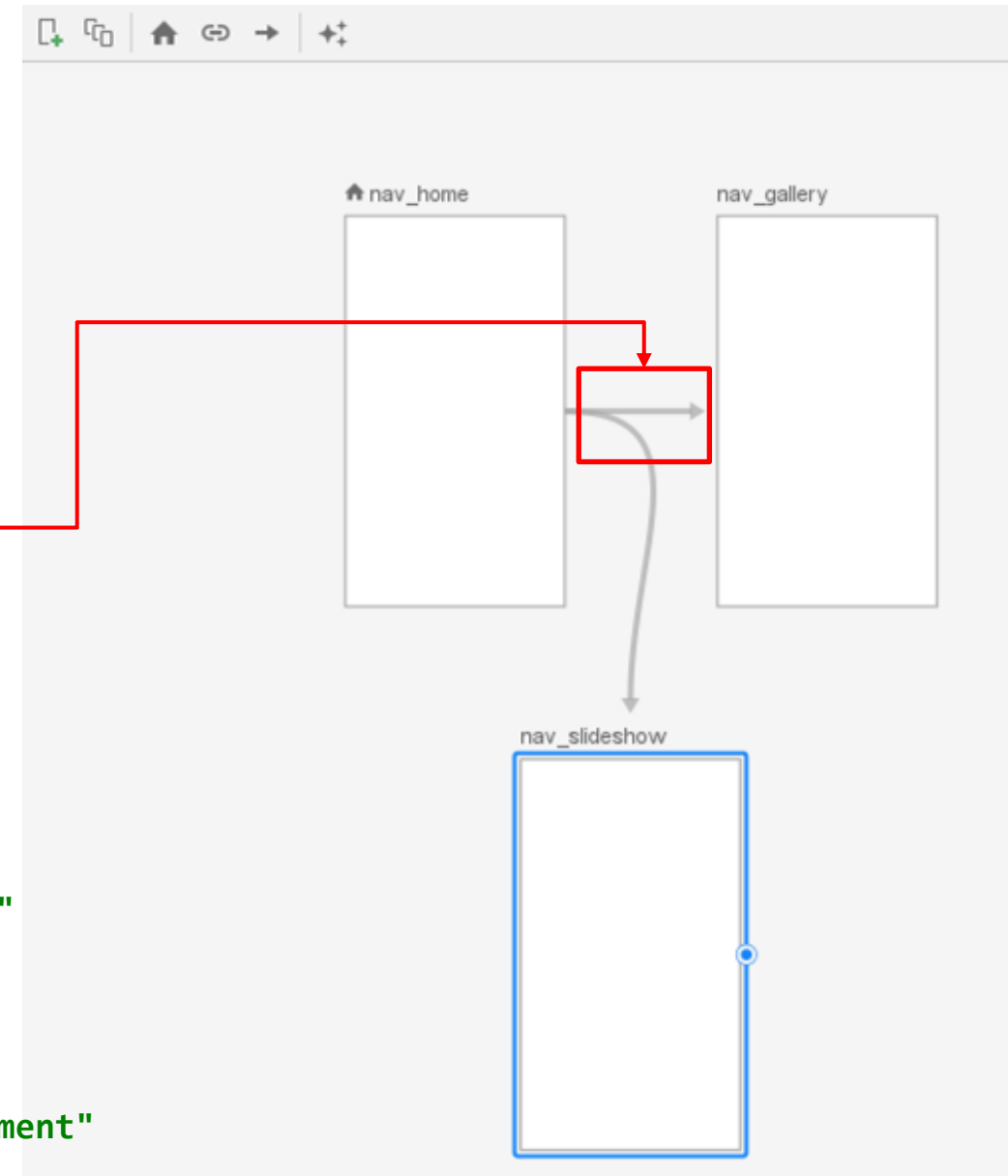
<?xml version="1.0" encoding="utf-8"?>
<navigation
  xmlns:android="http://schemas.android.com/apk/res/android"
  . . .
  app:startDestination="@+id/nav_home">

  <fragment
    android:id="@+id/nav_home"
    android:name="my.tarc.myapplication.ui.home.HomeFragment"
    . . .>
    <action
      android:id="@+id/homeToGallery"
      app:destination="@id/nav_gallery" />
    <action
      android:id="@+id/homeToSlideShow"
      app:destination="@id/nav_slideshow" />
  </fragment>

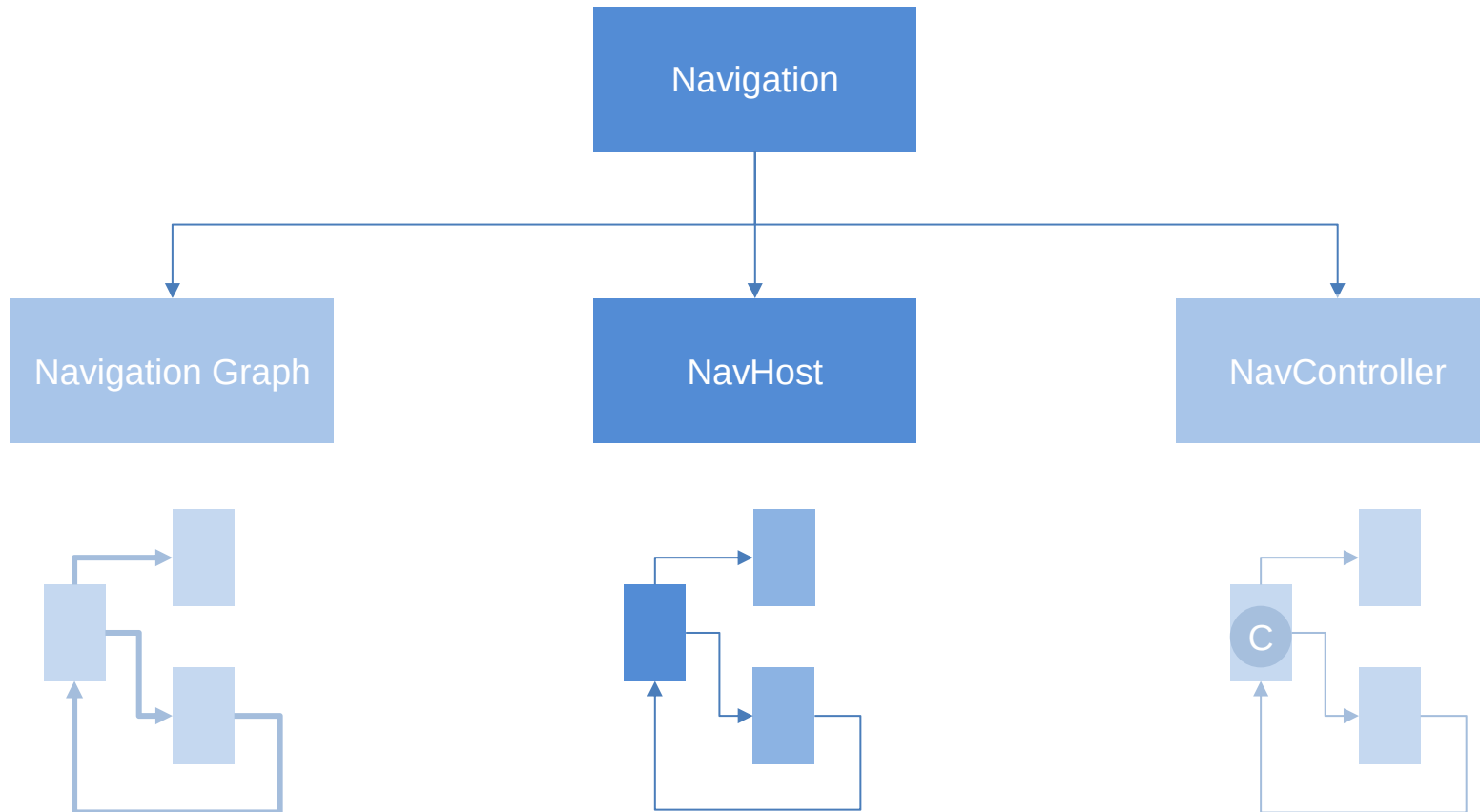
  <fragment
    android:id="@+id/nav_gallery"
    android:name="my.tarc.myapplication.ui.gallery.GalleryFragment"
    . . ./>

  <fragment
    android:id="@+id/nav_slideshow"
    android:name="my.tarc.myapplication.ui.slideshow.SlideshowFragment"
    . . ./>
</navigation>

```



Navigation



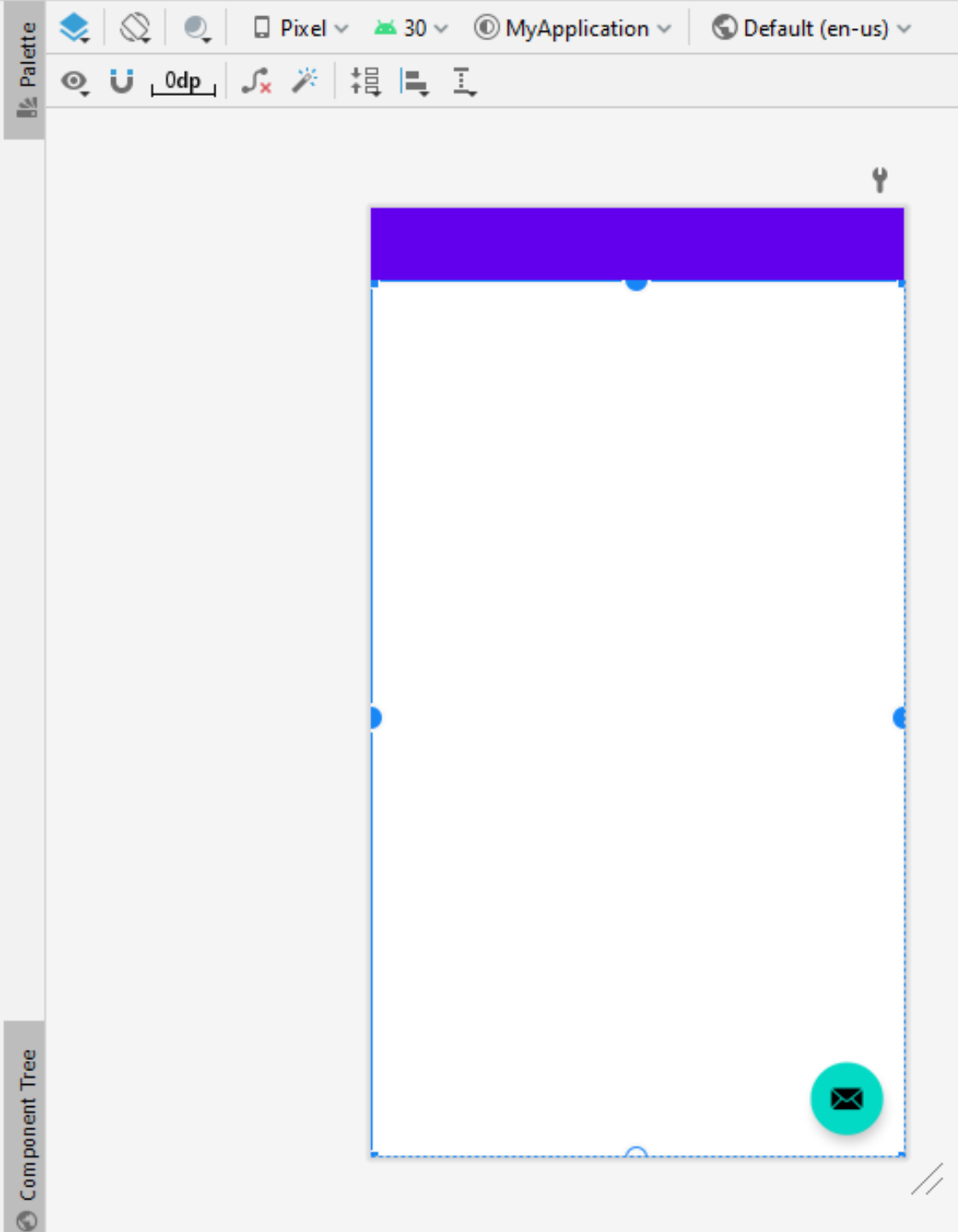
An empty container that displays destinations from your navigation graph.

```

1 <?xml version="1.0" encoding="Utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
3   xmlns:app="http://schemas.android.com/apk/res-auto"
4   xmlns:tools="http://schemas.android.com/tools"
5   android:layout_width="match_parent"
6   android:layout_height="match_parent"
7   app:layout_behavior="@string/appbar_scrolling_view_behavior"
8   tools:showIn="@layout/app_bar_main">
9
10  <fragment
11    android:id="@+id/nav_host_fragment_content_main"
12    android:name="androidx.navigation.fragment.NavHostFragment"
13    android:layout_width="match_parent"
14    android:layout_height="match_parent"
15    app:defaultNavHost="true"
16    app:layout_constraintLeft_toLeftOf="parent"
17    app:layout_constraintRight_toRightOf="parent"
18    app:layout_constraintTop_toTopOf="parent"
19    app:navGraph="@navigation/mobile_navigation" />
20 </androidx.constraintlayout.widget.ConstraintLayout>

```

Intercepts the system Back button. (Only one NavHost can be the default)

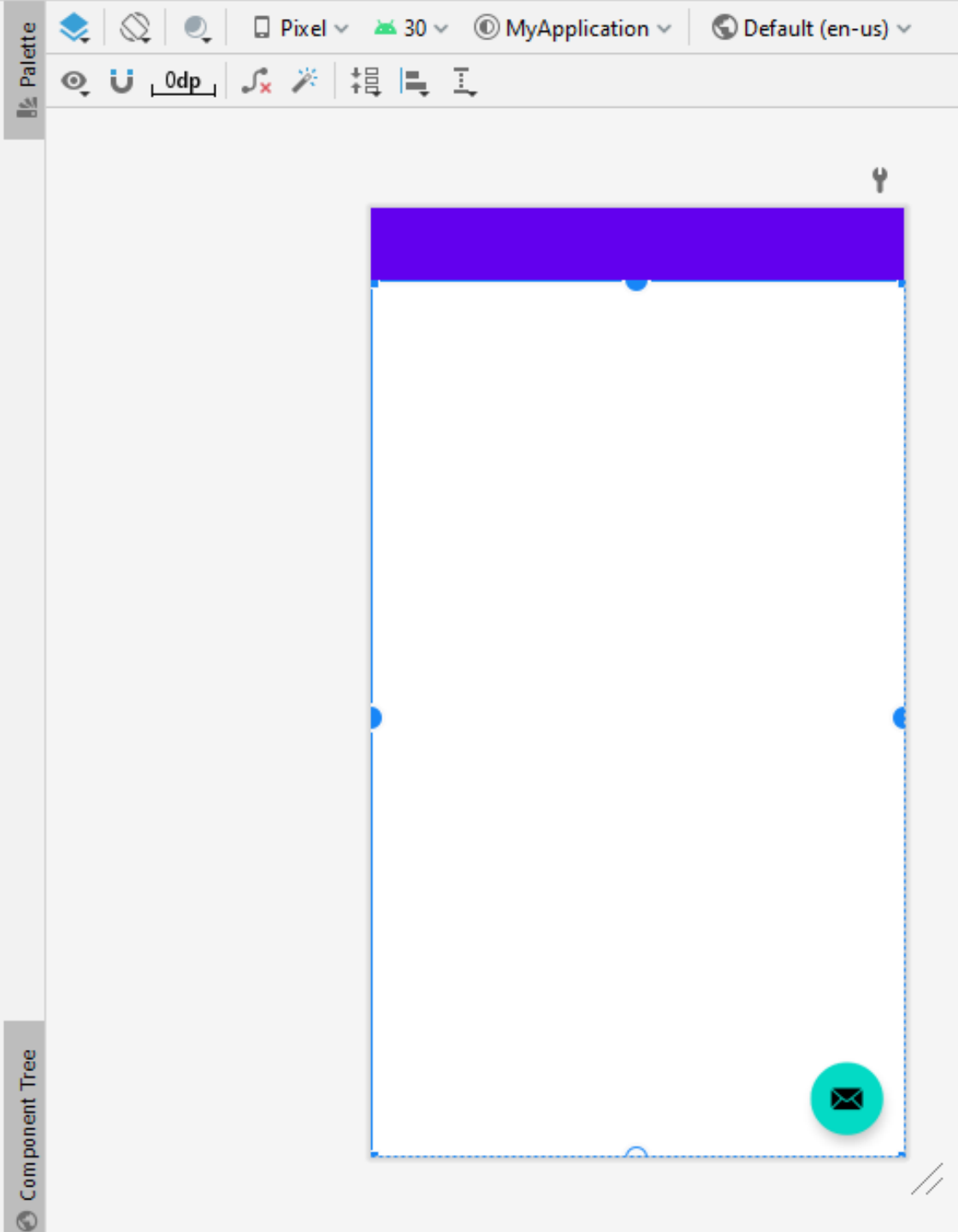


```

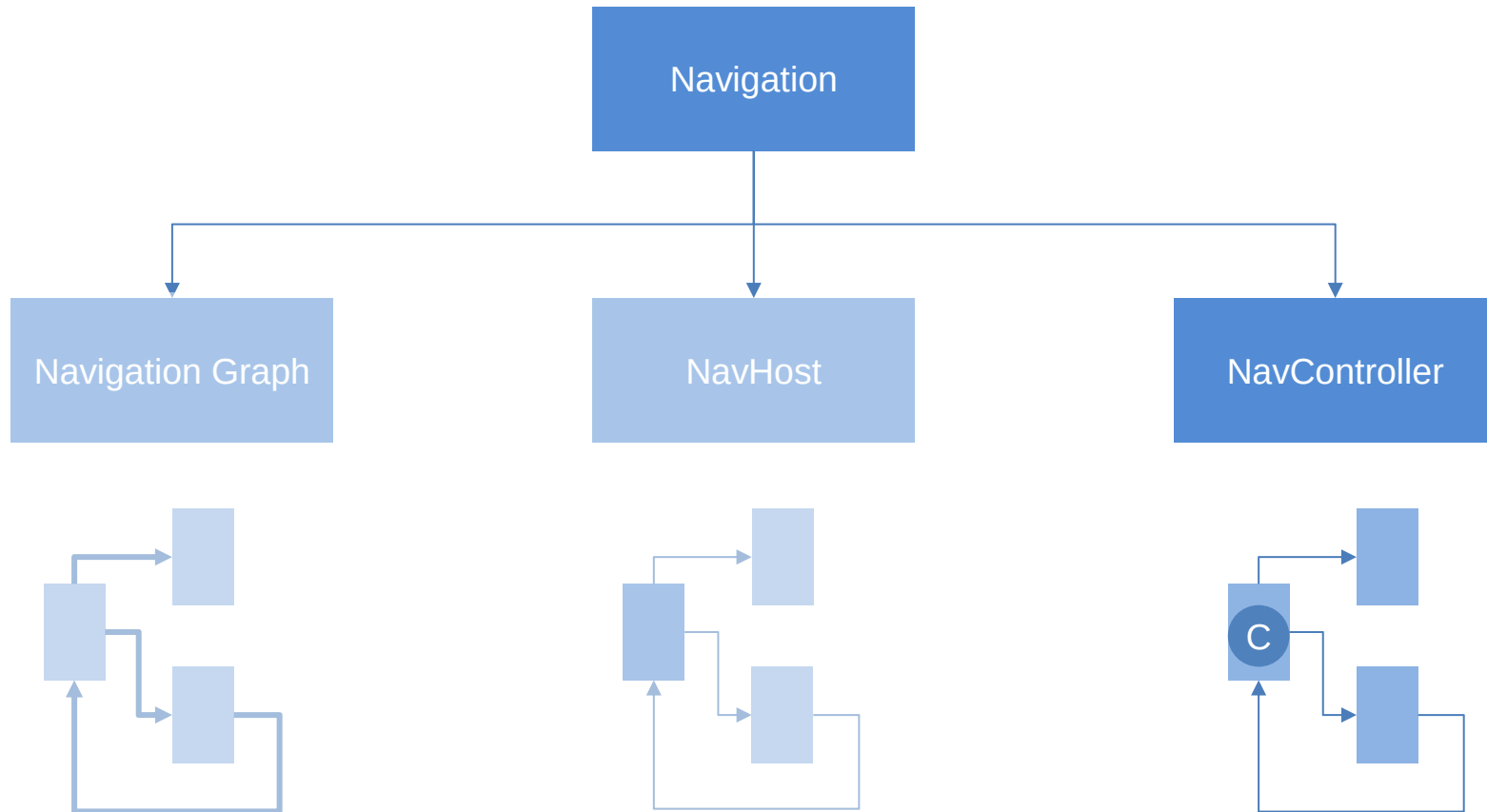
1 <?xml version="1.0" encoding="Utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
3   xmlns:app="http://schemas.android.com/apk/res-auto"
4   xmlns:tools="http://schemas.android.com/tools"
5   android:layout_width="match_parent"
6   android:layout_height="match_parent"
7   app:layout_behavior="@string/appbar_scrolling_view_behavior"
8   tools:showIn="@layout/app_bar_main">
9
10  <fragment
11    android:id="@+id/nav_host_fragment_content_main"
12    android:name="androidx.navigation.fragment.NavHostFragment"
13    android:layout_width="match_parent"
14    android:layout_height="match_parent"
15    app:defaultNavHost="true"
16    app:layout_constraintLeft_toLeftOf="parent"
17    app:layout_constraintRight_toRightOf="parent"
18    app:layout_constraintTop_toTopOf="parent"
19    app:navGraph="@navigation/mobile_navigation" />
20 </androidx.constraintlayout.widget.ConstraintLayout>

```

Link to the navigation graph



Navigation



An object that manages app navigation within a NavHost

Navigation Controller

Methods to get NavController:

```
Fragment.findNavController()  
View.findNavController()  
Activity.findNavController(viewId: Int)
```

Example:

```
val navController = findNavController(R.id.nav_host_fragment_content_main)  
  
navController.navigate(R.id.action_nav_home_to_profileFragment)
```

Navigation in Jetpack Compose

- Traditional Android navigation relies on XML and Activities
- Jetpack Compose Navigation simplifies transitions between screens

Navigation in Jetpack Compose – The Basic

- Navigation in Jetpack Compose is managed using the *android.navigation.compose* library.
- Key Concepts
 - NavHost
 - NavController
 - Composable() destinations

Navigation in Jetpack Compose – The Basic

- First step: Adding dependencies in gradle files (module level)

```
dependencies {  
    implementation (libs.androidx.navigation.compose)
```

Navigation in Jetpack Compose – The Basic

```
class MainActivity : ComponentActivity() {  
    @SuppressWarnings("UnusedMaterial3ScaffoldPaddingParameter")  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            NavigationTheme {  
                Scaffold(modifier = Modifier.fillMaxSize()) {  
                    MyAppNavHost(navController = rememberNavController())  
                }  
            }  
        }  
    }  
}
```

← Create NavHost

Navigation in Jetpack Compose – The Basic

```
@Composable
fun MyAppNavHost(navController: NavHostController) {
    NavHost(navController, startDestination = "home") {
        composable("home") { HomeScreen(navController) }
        composable("details") { DetailScreen(navController) }
    }
}
```

← Route / destination

Navigation in Jetpack Compose – The Basic

```
@Composable
fun HomeScreen(navController: NavHostController) {
    Column(modifier = Modifier
        .fillMaxSize(),
        verticalArrangement = Arrangement.Center, // Center vertically
        horizontalAlignment = Alignment.CenterHorizontally // Center horizontally
    ) {
        Button(onClick = {
            Log.i("Navigation", "Clicked")
            navController.navigate("details") }) { ←
            Text("Go to Details")
        }
    }
}

@Composable
fun DetailScreen(navController: NavHostController) {
    Column(modifier = Modifier
        .fillMaxSize(),
        verticalArrangement = Arrangement.Center, // Center vertically
        horizontalAlignment = Alignment.CenterHorizontally // Center horizontally
    ) {
        Button(onClick = { navController.popBackStack() }) {
            Text("Go Back")
        }
    }
}
```

Navigation in Jetpack Compose – Navigate with data

```
composable("details/{itemId}") { backStackEntry ->  
    val itemId = backStackEntry.arguments?.getString("itemId")  
    DetailScreen(navController, itemId)  
}
```

```
navController.navigate("details/123")
```

Navigation in Jetpack Compose – Bottom navigation

```
@Composable
fun BottomNavigationBar(navController: NavController) {
    BottomNavigation {
        BottomNavigationItem(
            icon = { Icon(Icons.Default.Home, null) },
            label = { Text("Home") },
            selected = false,
            onClick = { navController.navigate("home") }
        )
    }
}
```

Question?

1. What is the main difference between a Fragment and an Activity?
2. What are the advantages of using Fragment compared to Activity?

Create Dialog

Dialogs

Use dialog for:

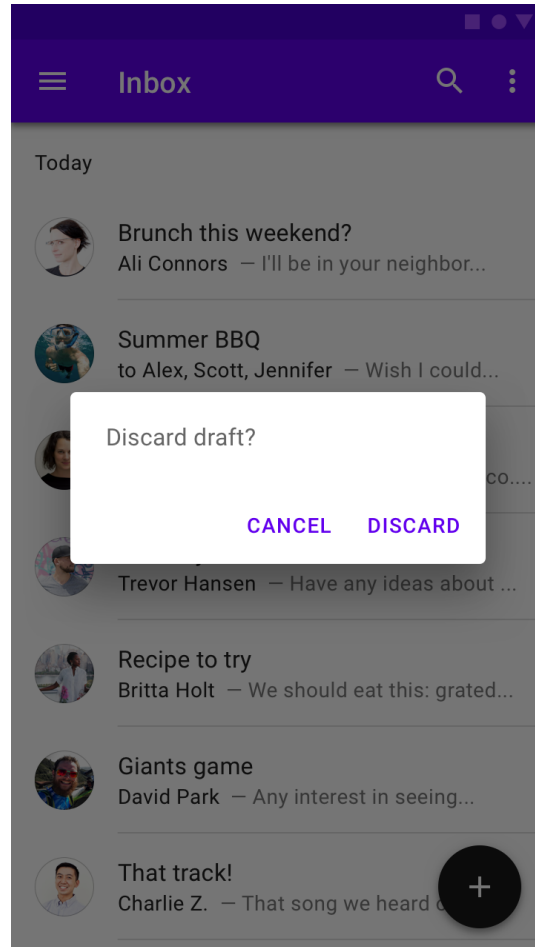
1. critical information
2. make decisions
3. encapsulate multiple tasks within a discrete process

Use it sparingly because they are interruptive in nature

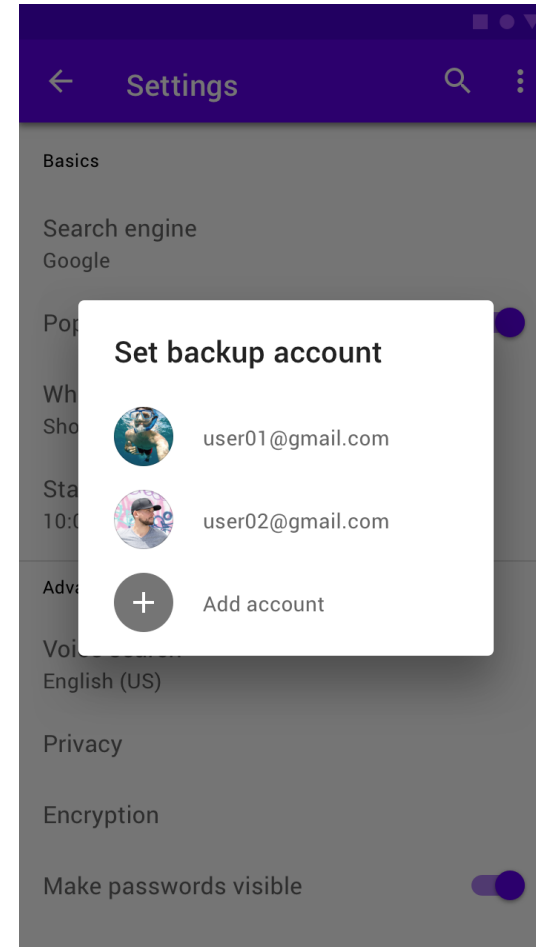
Dialog vs Notification and Snackbar

Component	Priority	User Action
Snackbar	Low	Optional It disappears automatically
Notification	Medium	Optional It remains until dismissed by the user, or if the state that caused the notification is resolved
Dialog	High	Required It blocks app usage until the user takes a dialog action or dismisses the dialog

Dialogs

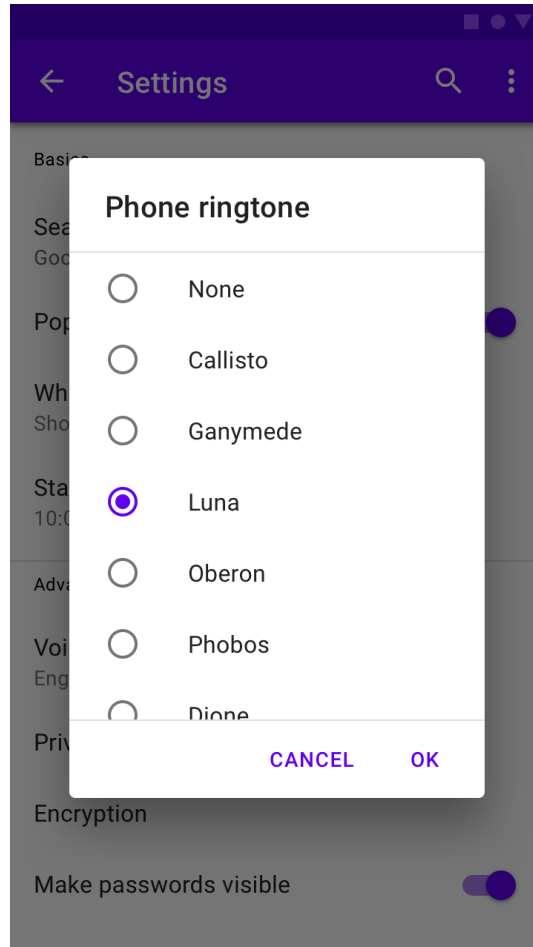


1. Alert Dialog

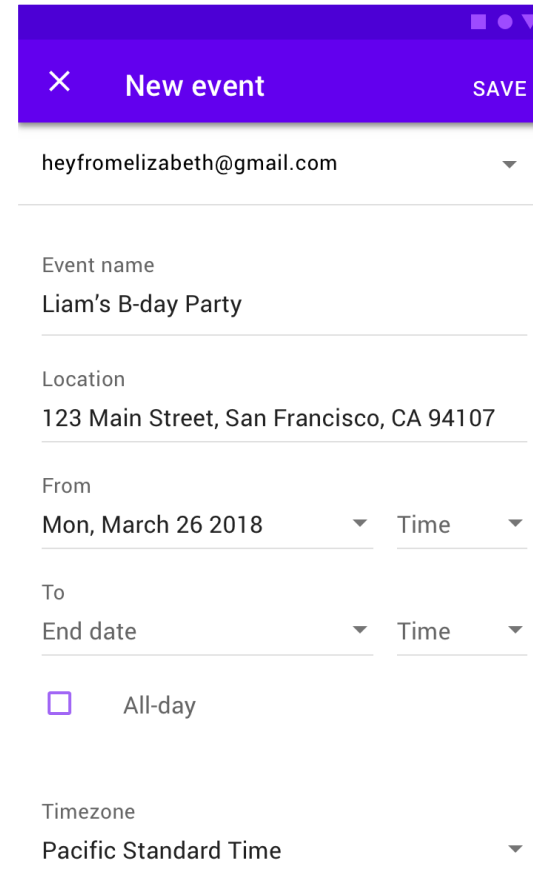


2. Simple Dialog

Dialogs



3. Confirmation Dialog



4. Full screen Dialog

Dialogs

Avoid:

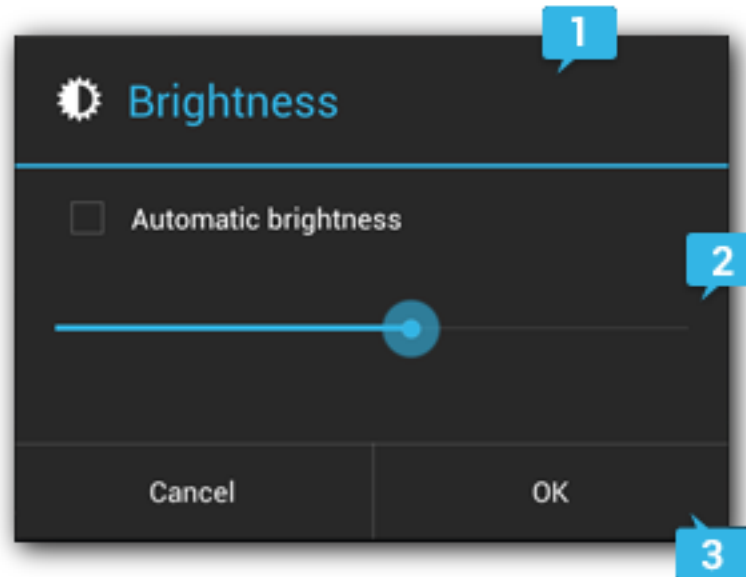
- Open additional dialogs
- Contain scrolling content, particularly alerts

Exceptions include:

- Full-screen dialogs may open additional dialogs, such as pickers, because their design accommodates additional layers of material without significantly increasing the app's perceived z-depth or visual noise.

AlertDialog

1. Title (optional)
2. Content area: This can display a message, a list, or other custom layout.
3. Action buttons: There should be no more than three action buttons in a dialog.

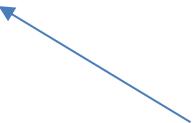


AlertDialog

```
@Composable
fun SimpleAlertDialog(showDialog: Boolean, onDismiss: () -> Unit) {
    if (showDialog) {
        AlertDialog(
            onDismissRequest = { onDismiss() },
            title = { Text(text = "Alert") },
            text = { Text("Are you sure you want to proceed?") },
            confirmButton = {
                Button(onClick = { onDismiss() }) {
                    Text("OK")
                }
            },
            dismissButton = {
                Button(onClick = { onDismiss() }) {
                    Text("Cancel")
                }
            }
        )
    }
}
```

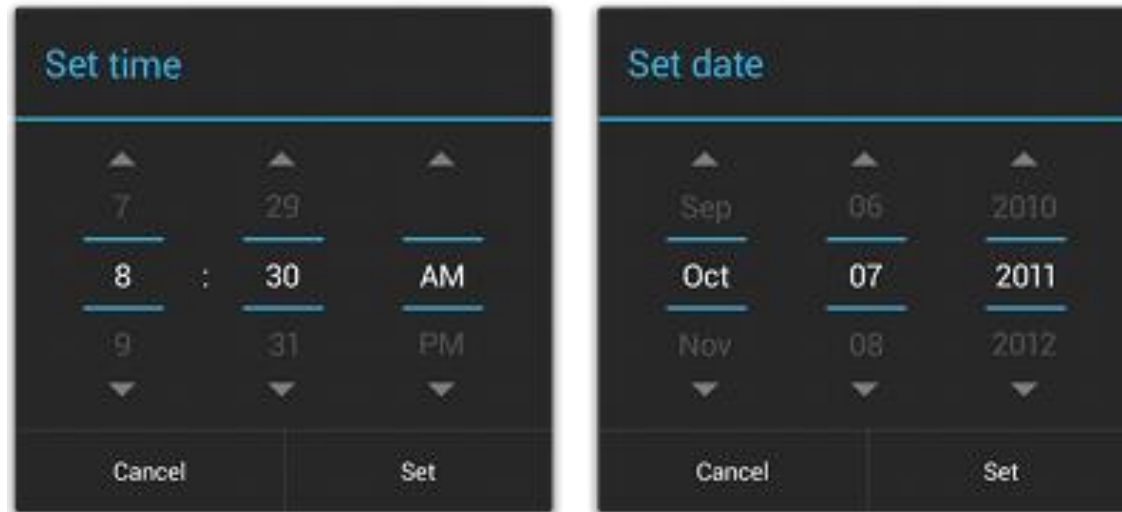
```
@Composable
fun MainScreen() {
    var showDialog by remember {
        mutableStateOf(false)
    }

    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment =
            Alignment.CenterHorizontally
    ) {
        Button(onClick = { showDialog = true }) {
            Text("Show Dialog")
        }
        SimpleAlertDialog(showDialog) { showDialog
            = false }
    }
}
```



Pickers

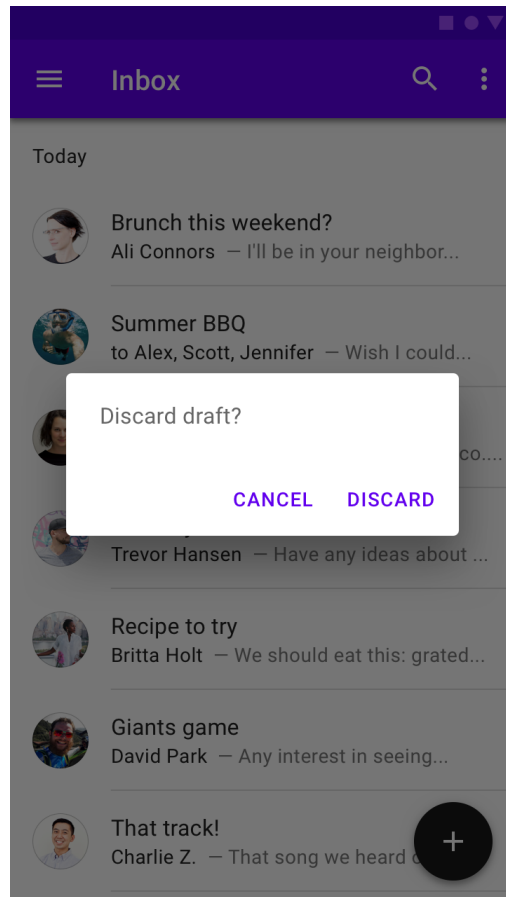
Pick a time or date



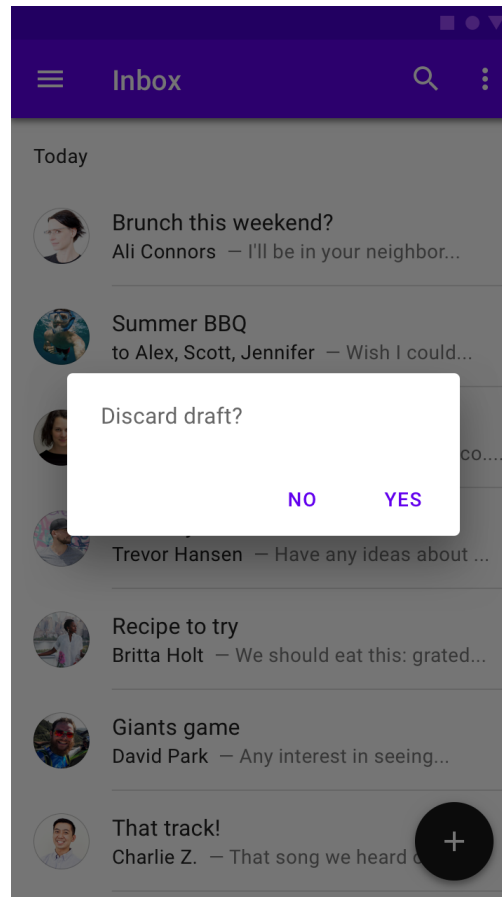
Question?

1. Which AlertDialog is better? Why?

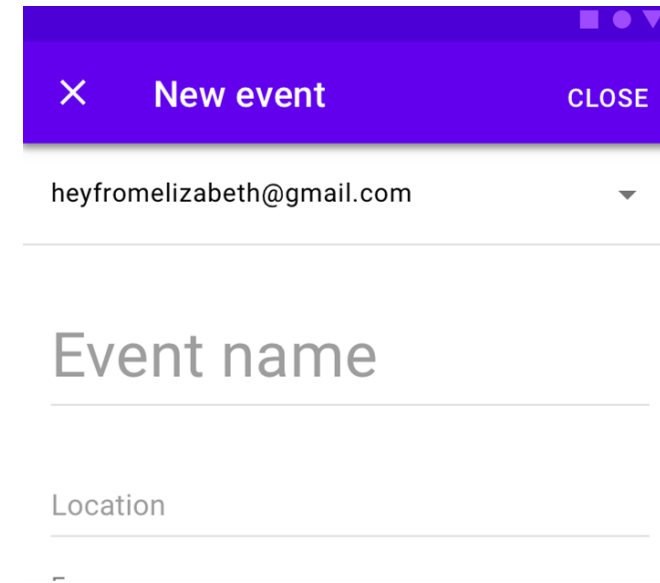
A



B



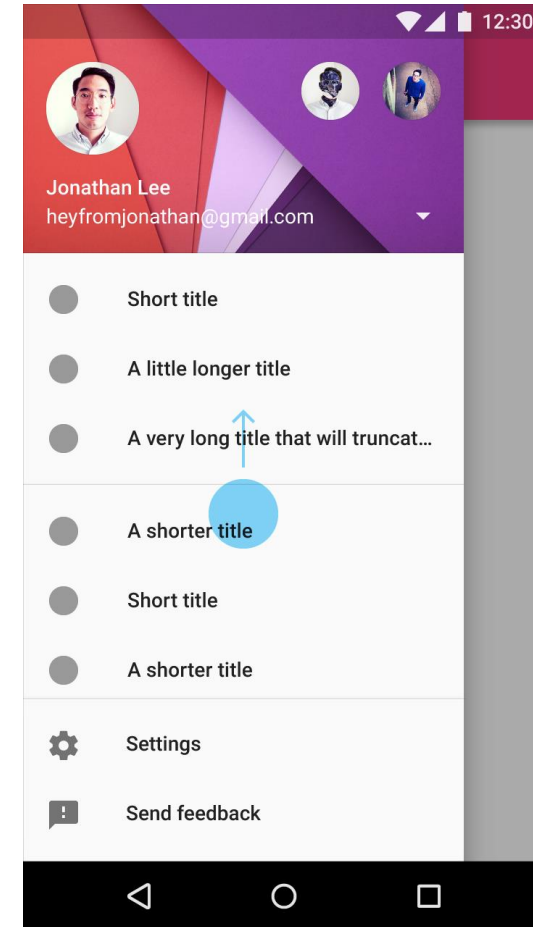
2. Evaluate the following full screen dialog:



Create Navigation Drawer

Navigation Drawer

Displays the app's main navigation options

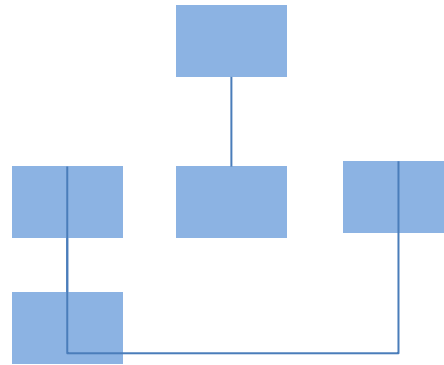


Navigation Drawer

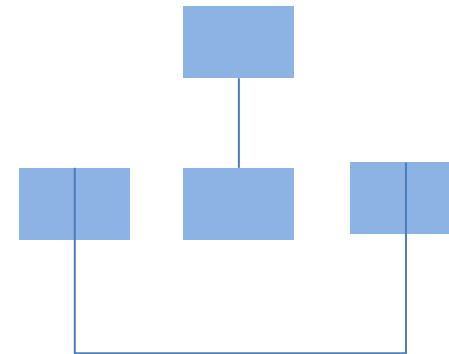
Recommended for:



≥ 5 top-level destinations



≥ 2 levels of navigation hierarchy



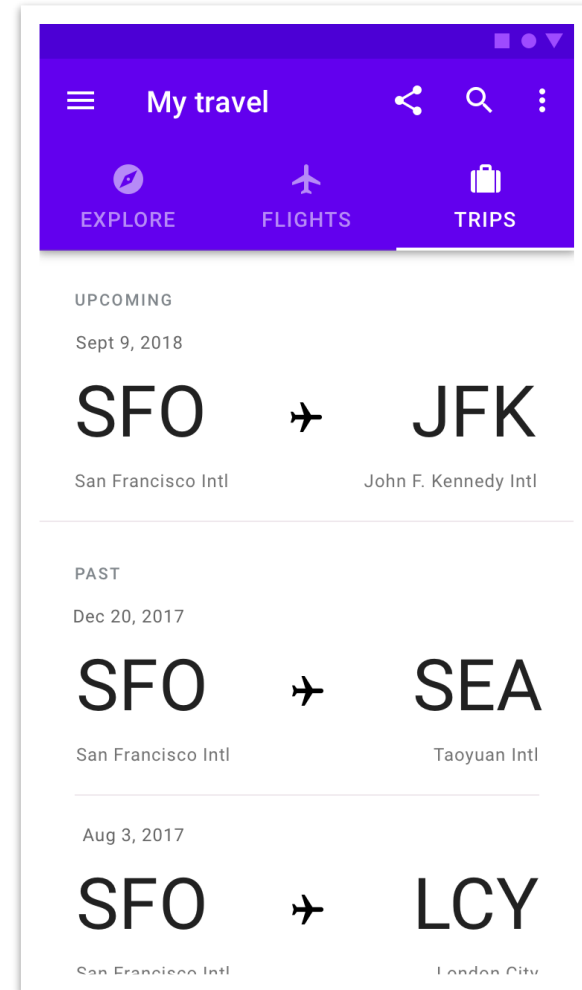
Quick navigation
between unrelated
destinations

Create Tabs

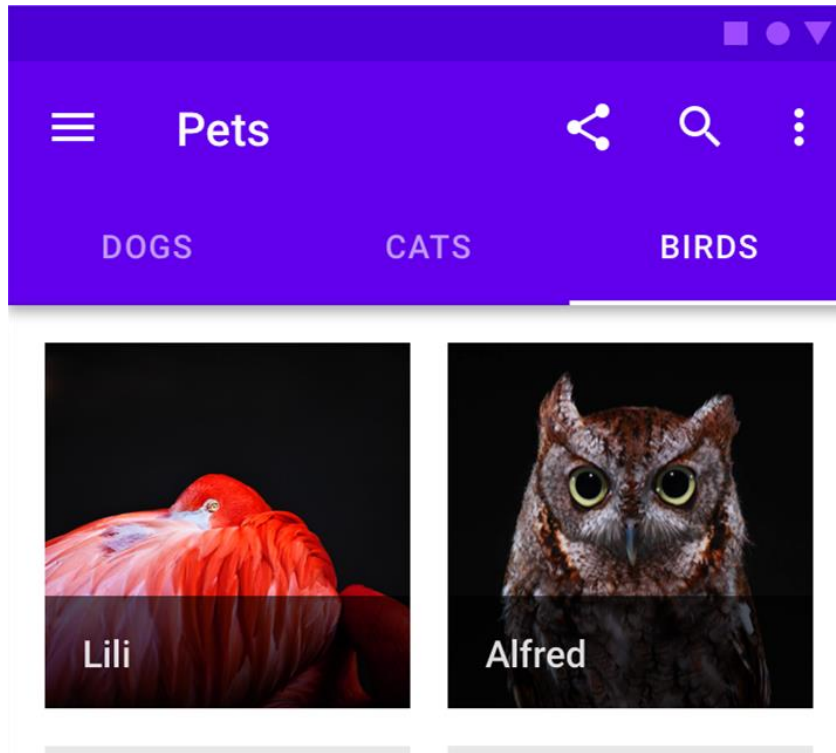
Tabs

It organizes and allows navigation between groups of content

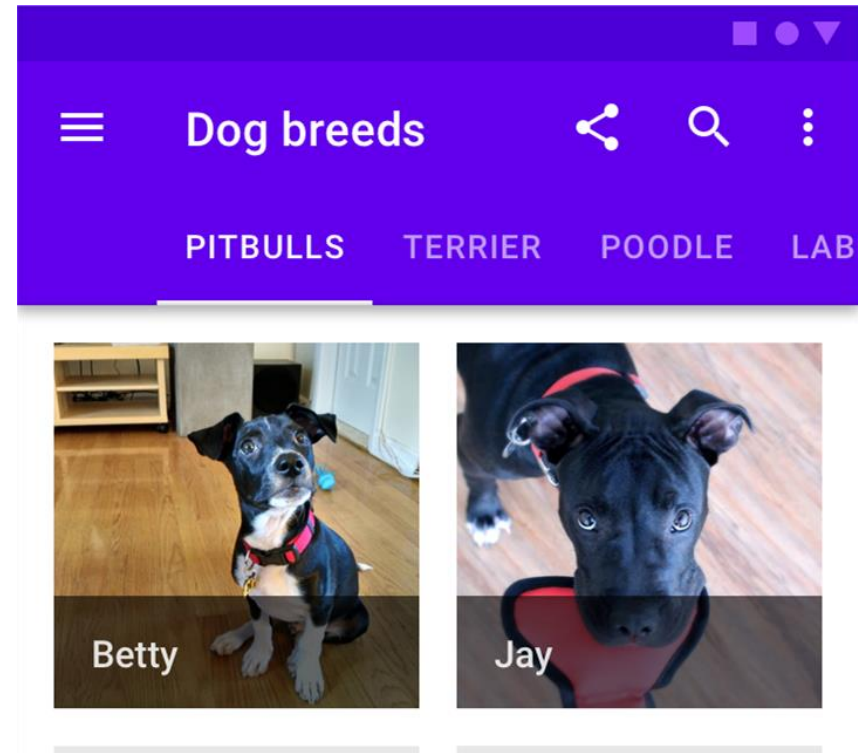
Content should be related and at the same level of hierarchy



Tabs



1. Fixed Tabs



2. Scrollable Tabs

Question?

1. Compare Navigation Drawer and Tabs
2. Among Navigation Drawer and Tabs, which UI is suitable for each of the following apps:
 - a. A news app
 - b. A weather app
 - c. A contact app

Conclusion

Understand navigation principles

Build a dynamic and multi-pane UI using Fragment

Create Navigation Component

Create Dialog

Create Navigation Drawer

Create Tabs